Common Ground Crowdfunding

Financial Decentralized Application - Project Report
Prepared from the project source on 13 October 2026

1. Introduction

Common Ground is a single-campaign Ethereum crowdfunding application. Its demonstration campaign, "The neighborhood learning room", presents a community space for reading, learning and sharing ideas. Contributors connect MetaMask and send test ETH to a Solidity contract. The contract holds funds until the campaign deadline and then permits either beneficiary withdrawal or individual refunds, depending on whether the funding goal was reached.

A decentralized application combines a conventional web interface with blockchain execution. Ethereum records transactions and runs the campaign rules; the browser provides a usable interface; Flask and SQLite maintain a convenient transaction-history index. The backend does not hold campaign funds or sign wallet transactions. This division makes the contract the authority for financial settlement while preserving familiar web application features.

2. Problem Statement

Community fundraising requires contributors to trust that an organizer will apply the advertised funding conditions. If a campaign raises too little to proceed, returning contributions can require manual administration. Contributors also need clear evidence that their payments were recorded.

This project addresses those concerns through an all-or-nothing funding rule enforced on-chain. Once deployed, the beneficiary, goal and deadline cannot be changed. Successful campaigns release funds only after the deadline; unsuccessful campaigns allow contributors to reclaim their recorded ETH. This guarantee concerns settlement conditions, rather than how the beneficiary spends funds after withdrawal. Gas fees remain the responsibility of each transaction sender.

3. Objectives

- Implement an Ethereum contract for contributions, withdrawal and refunds.
- Integrate MetaMask without collecting private keys or seed phrases.
- Present live funding progress, campaign status and wallet contributions.
- Verify transaction hashes independently before recording history.
- Validate inputs and communicate wallet, network and service errors.
- Provide automated test sources and repeatable deployment instructions.

Scope and evidence

This report describes the checked-in implementation and test sources. Automated tests were not executed for this report. Live Sepolia deployment, browser interactions, screenshots and performance measurements were not independently verified. The project is an educational demonstration using test ETH.

System Architecture and Technologies

4. System Architecture
The application has four cooperating layers:

Browser UI -> MetaMask -> Ethereum RPC -> Crowdfund contract
Browser UI -> Flask API -> independent Ethereum RPC verification
Flask API -> SQLite transaction-history database

The frontend reads campaign state through the wallet provider and requests a signature when a user contributes, withdraws or claims a refund. Ethereum executes the transaction. The browser then submits the transaction hash to Flask. Flask uses its own configured RPC provider to obtain authoritative transaction and receipt data, and stores the result in SQLite.

The separation matters because a database write cannot establish that ETH changed hands. Blockchain state determines the financial outcome. Conversely, a confirmed blockchain transaction remains valid even when the history service is unavailable. The interface retains unresolved hashes and retries indexing while the wallet is connected.

The frontend and backend must use the same chain ID and deployed contract address. Local development uses chain ID 1337; the documented public-testnet workflow uses Sepolia, chain ID 11155111. Wallet account and network changes reset the frontend session and require reconnection. Session checks reduce the chance of displaying asynchronous results for an outdated account.

5. Technologies Used
- Next.js 16.3.7 and React 19.2.8: page composition and interactive UI.
- TypeScript: typed campaign, wallet and transaction representations.
- ethers.js 6: wallet provider, contract calls and exact ETH/wei conversion.
- Solidity: immutable campaign rules and ETH custody.
- Flask 3.1.2 and Web3.py 7.13.0: API and RPC verification.
- SQLite: lightweight persistence of public transaction metadata.
- Ganache: local blockchain development and contract test execution.
- Docker Compose: coordinated frontend/backend deployment.
- Gunicorn: production backend serving in the container configuration.

Versions above reflect package declarations, rather than an independently inspected running deployment. The contract compiler script enables optimization with 200 runs and targets the Shanghai EVM. The Solidity source declares pragma ^0.8.30.

Technology rationale
The stack separates presentation, verification and settlement into small components. SQLite is appropriate for a modest single-instance educational service. MetaMask provides user-controlled signing, while Solidity makes the campaign conditions inspectable and executable without relying on the Flask server to release funds.

Smart Contract Design

6. Smart Contract Design
The implementation is sol/Crowdfund.sol. Its constructor accepts a beneficiary address, target in wei and duration in seconds. It rejects a zero beneficiary, zero goal or zero duration. The deadline is calculated from the deployment block timestamp. Beneficiary, goal and deadline are immutable.

State includes totalRaised, withdrawn and a contributions mapping keyed by wallet address. totalRaised represents cumulative historical contributions, rather than the remaining ETH balance. It is deliberately not reduced after settlement. A wallet's contribution is reset to zero when its refund succeeds.

Contribution
contribute() is payable. It accepts positive ETH only when block.timestamp is strictly before the deadline. It increases the sender's contribution and totalRaised, then emits Contributed. Overfunding is allowed; reaching the goal early does not end the campaign or unlock early withdrawal.

Successful settlement
withdraw() requires the caller to be the beneficiary, the deadline to have passed, totalRaised to meet the goal and withdrawn to be false. It marks the campaign withdrawn before transferring the entire contract balance to the beneficiary. A failed transfer reverts the transaction, including the state update. Withdrawn records the recipient and amount.

Unsuccessful settlement
refund() requires the deadline to have passed and totalRaised to remain below the goal. The caller must have a positive recorded contribution. The function clears that contribution before transferring its value back and emits Refunded. Contributors claim individually; the contract does not loop over all donors. A second successful refund from the same contribution is prevented.

Security and gas considerations
Authorization depends on msg.sender, not a frontend-supplied identity. Outgoing transfers follow checks-effects-interactions: settlement eligibility is checked and state updated before external calls. These choices address repeat settlement and basic reentrancy concerns, but do not constitute a comprehensive audit. Ordinary direct ETH transfers without a supported payable function are rejected because no receive or fallback handler is defined.

Immutable campaign parameters reduce administrative discretion. Per-wallet refunds avoid a settlement loop whose gas cost grows with donor count. Compiler optimization is enabled, but gas savings were not measured. ETH amounts use integer wei, avoiding floating-point accounting errors. The application never promises that network fees are refundable.

State transitions
Active -> deadline reached -> goal met -> beneficiary withdraws once.
Active -> deadline reached -> goal missed -> each donor claims a refund.
No contract path changes the campaign goal or extends the deadline.

Application Design and Implementation

7. Application Design
The interface combines a campaign story, funding panel and transaction history. The story explains the community purpose and settlement promise. The funding panel shows ETH raised, goal, percentage funded, deadline and the connected wallet's contribution. It supports manual amounts and preset contributions of 0.01, 0.05 and 0.1 ETH.

Buttons reflect the campaign state. Contributions close after the deadline. Refunds appear for eligible contributors when the goal was missed. Withdrawal appears for the beneficiary after a successful campaign ends. These controls help users choose valid actions, while the contract remains responsible for enforcing permissions.

8. Implementation
frontend/src/hooks/use-campaign.ts coordinates wallet connection, contract reads, transaction submission and history reconciliation. It reads beneficiary, goal, total raised, deadline, withdrawn status, the user's contribution and the latest block. Settlement availability is based on blockchain time. A 12-second refresh interval updates campaign and history data while connected.

Before submission, the frontend checks the configured network, selected signer and presence of contract code at the address. contributionWei() accepts a positive decimal ETH amount with at most 18 decimal places and converts it to bigint wei through ethers. Invalid, zero, negative and over-precision amounts are rejected. Wallet rejection receives an actionable message.

After broadcasting, the UI exposes the transaction hash, attempts history indexing and waits for one block confirmation with a 120-second timeout. An error after broadcast prompts the user to check the hash before retrying. This distinction helps prevent duplicate contributions caused by confusing an indexing failure with a payment failure.

Backend verification
POST /api/transactions accepts a syntactically valid transaction hash. Flask checks the RPC chain ID, recipient contract and exact four-byte selector for contribute(), refund() or withdraw(). Sender, ETH value and receipt status come from RPC, rather than client claims. Unknown transactions return 404, invalid or unrelated requests return 400, and configuration/provider failures return 503.

GET /api/transactions?wallet=... returns up to 100 indexed records for the wallet, configured chain and contract. GET /api/health reports service liveness. CORS allows the configured web origin; incoming request bodies are limited to 2,048 bytes.

SQLite stores hash, chain ID, contract address, wallet, action, transaction value in wei, status, block number and creation timestamp. The primary key is (chain_id, hash), enabling idempotent status updates. Transaction value is stored as text; for withdrawal/refund calls, this is the call's attached ETH value, not the contract's outgoing settlement amount.

Deployment, Testing and Results

Deployment implementation
Docker Compose builds two services, exposes the frontend on localhost:3000 and the backend on localhost:5001, and stores SQLite in a named volume. Health checks support startup coordination. Application liveness does not establish blockchain connectivity. Docker starts the web services; contract deployment remains a separate operation.

The local deployment script creates a campaign with a 1 ETH goal and a one-hour duration. The README also describes Remix and MetaMask deployment to Sepolia, including a 0.002 ETH example goal. Public frontend settings are embedded at build time, so changes to address, chain or API URL require rebuilding the frontend.

A Render Blueprint defines a Docker frontend and a backend with persistent SQLite storage. This is deployment configuration, rather than evidence that hosted services are live. RPC credentials belong in backend environment settings. Wallet private keys are unnecessary for running either application service.

9. Testing and Results
The repository contains automated tests at three levels:
- Frontend unit tests: precise ETH parsing, invalid amounts, wallet rejection, missing wallets, requested network switching and failed network switching.
- Contract tests: constructor validation, positive contributions, zero-value rejection, premature settlement rejection, beneficiary-only withdrawal, deadline enforcement and prevention of repeat withdrawals/refunds.
- Backend tests: authoritative RPC data, database persistence, malformed requests, pending/confirmed/failed status transitions, unrelated contract/method rejection, RPC/network errors, CORS and configuration checks.

Contract tests use an in-process Ganache EVM. Backend tests use temporary SQLite databases and mocked RPC responses. These approaches isolate financial rules and API behavior without requiring a deployed Sepolia contract.

Results and evidence boundaries
Source review confirms that the implementation and these test scenarios exist. No fresh pass/fail counts are claimed: test suites, lint and build were not run during preparation of this report. The README records an earlier successful production frontend build during Docker deployment; this is repository documentation, not a newly reproduced result. Browser/MetaMask interaction and Sepolia deployment remain unverified here.

Manual acceptance evidence still needed
Capture wallet connection, an approved contribution, explorer receipt and history entry. Use separate campaigns to demonstrate successful withdrawal and failed-goal refund. Confirm unauthorized settlement and repeated claims fail. Restart the backend to check persistence, and interrupt history service during a payment to check recovery without a second payment. Record desktop/mobile views and invalid-input behavior. No screenshots are fabricated or included as observed results.

Evaluation, Challenges and Limitations

10. Challenges Encountered
The code reveals several integration challenges and the strategies used to address them. These observations describe implementation choices; they are not a reconstructed development diary.

Exact currency handling: JavaScript floating-point arithmetic is unsuitable for wei accounting. The project uses decimal validation and bigint conversion, preserving up to 18 ETH decimal places.

Wallet and network consistency: users can change accounts or chains during asynchronous requests. Event handlers reset the session, and submission checks confirm the expected signer and network. Users reconnect to reload the campaign.

Blockchain/API consistency: a transaction can be broadcast before the server RPC sees it, or can succeed while Flask is unavailable. The frontend saves unresolved hashes locally and retries indexing. The backend accepts repeated submissions idempotently and derives status from receipts.

Deployment consistency: the wallet and server use separate RPC access, and frontend configuration is fixed during the build. Documentation emphasizes matching chain/address settings and rebuilding after public configuration changes. Persistent volumes or disks preserve SQLite history across container replacement.

11. Limitations
Only one configured campaign is presented at a time. The story is a demonstration, and there is no campaign directory, creation interface, organizer verification or milestone-based release. The contract controls funding conditions but cannot verify whether a beneficiary delivers the promised community project.

History is an index of submitted hashes, not a complete blockchain scan. Transactions made outside the app need explicit indexing. History responses are limited to 100 rows. Receipt status is refreshed through POST reconciliation, rather than by the GET endpoint itself. Clearing browser storage can remove unresolved hashes unless the user preserves them elsewhere.

One mined receipt is treated as confirmed, not finalized. Chain reorganizations, replaced transactions and dropped transactions require additional reconciliation. The application has no background indexer, measured load capacity or independently audited contract. No rate-limiting layer is implemented.

Security, privacy and performance evaluation
MetaMask protects signing keys from the application. Public wallet addresses and transaction metadata remain visible; pseudonymous addresses should not be described as anonymous identities. CORS restricts browser origins but is not authentication. The API intentionally permits anyone to index an existing public campaign transaction.

Twelve-second polling, 10-second backend RPC timeouts and 12-second frontend API timeouts are configuration choices, not measured response-time results. Gas costs, usability outcomes and end-to-end latency have not been benchmarked. SQLite and a single-instance backend suit the current scale but need reassessment for broader deployment.

Future Improvements and Conclusion

12. Future Improvements
First, strengthen evidence and operational confidence. Execute the supplied tests and record their outputs, then perform the complete MetaMask/Sepolia walkthrough. Add real screenshots and deployment transaction references to the submission. Measure confirmation latency, API response times and gas costs across contribution, withdrawal and refund scenarios.

Second, improve reconciliation. A background event indexer could discover transactions independently of browser submissions and update receipt status. Confirmation-depth tracking, reorganization handling and replacement-transaction detection would provide more reliable status semantics. Pagination would remove the 100-record history ceiling.

Third, expand the product carefully. A campaign factory and directory could support multiple campaigns, with explicit organizer information and configurable campaign narratives. Any milestone release or beneficiary governance feature would require a revised contract design and new financial-rule tests. Independent security review should precede use with real funds.

Fourth, strengthen hosting and accessibility. Add database backups, API rate limits, monitoring and RPC failure alerts. Evaluate keyboard navigation, screen-reader output, smaller displays and wallet flows with users. Public deployment should use HTTPS and maintain durable storage. Additional automated tests should target failed transfers, adversarial receiver contracts and recovery under network interruption.

13. Conclusion
Common Ground implements the central elements of a financial DApp: a Solidity contract, MetaMask connection, ETH transactions, a web interface, a Flask backend and transaction-history persistence. Its all-or-nothing model makes the settlement conditions explicit and enforces them on Ethereum. Contributors can fund a campaign before its deadline; afterward, successful campaigns allow beneficiary withdrawal and unsuccessful campaigns allow individual refunds.

The design keeps financial authority on-chain and uses the backend as a verified public-data index. Integer currency handling, immutable campaign parameters and state updates before outgoing transfers provide a sound educational foundation. The main remaining work is demonstrated test execution, live wallet evidence and stronger indexing and operational controls.

Source basis
This report was prepared from doc/assignment.docx, README.md, sol/Crowdfund.sol, frontend/package.json, frontend/src/hooks/use-campaign.ts, frontend/src/lib/, frontend/src/components/, frontend/scripts/compile.mjs, frontend/tests/, backend/src/, backend/tests/test_app.py, compose.yaml and render.yaml. These local sources support the implementation claims; external documentation and live deployment state were not independently verified.

Submission note
The PDF contains seven pages and follows all thirteen report sections requested by the assignment. Student identity details, real execution screenshots and the separate 5-8 minute demonstration video should be supplied by the author. This report does not claim those artifacts already exist.